Bags, Vectors & Transformers

Web Scraping and APIs · Bonus, Part 3

Denise J. Roth, PhD Candidate
Strategic Communication Group · Wageningen University & Research

Two Ways to Collect Text from the Web

APIs: the front door

· the site hands out data in a structured format (JSON)
· clear rules on how much you may take
· little code, rarely breaks
· **always the first choice** when one exists

Scraping: the fallback

· download the web pages, pick the information out of the HTML yourself
· works on almost any site
· fragile: breaks when the site changes
· more legal and ethical questions

Before both: **does a ready-made dataset already exist?** Data archives, replication files and the university library (Nexis Uni for news, Delpher for Dutch newspapers) often save weeks of work.

Before You Collect Anything

question	why it matters
Is there an API?	structured data, clear rules, far less work
Terms of service?	many sites forbid automated collection; breaking the terms can be a contract issue
`robots.txt`?	the site's machine-readable wishes for bots: not law, but hard to defend ignoring
Personal data?	names, usernames and opinions are personal data under the GDPR, even when public
Copyright?	EU research organizations may mine texts (Art. 3, 2019 Copyright Directive), but generally not republish them: share counts and IDs, not full texts
Burden on the server?	small sites can be overwhelmed by careless scripts

Not legal advice; rules differ by country and platform. Talk to your data steward or privacy officer early. They see these questions all the time.

HTTP, Politely

Your code sends a **request**; the server returns a **response**: a status code, headers, and a body (HTML or JSON).

200	OK
403	forbidden
404	not found
429	too many requests
5xx	server error

A well-behaved script

- **identifies itself**: a `User-Agent` with your name and e-mail
- uses a **timeout**
- **waits** between requests
- **retries** after 429 or 5xx, waiting longer each time, and honors `Retry-After`

```
session.headers["User-Agent"] = "BVT-course/1.0 (contact:
you@wur.nl)"
r = session.get(url, params=params, timeout=20)
```

Reading robots.txt

```
User-agent:  *
Disallow:  /search
Disallow:  /account/
Crawl-delay:  5

User-agent:  GPTBot
Disallow:  /
```

A made-up news site. Many real news sites now block AI crawlers by name.

Python's built-in `RobotFileParser` answers "may I?" for any URL:

```
rp.can_fetch("BVT-course", ".../politics/budget") # True
rp.can_fetch("BVT-course", ".../search?q=x") # False
rp.can_fetch("BVT-course", ".../searchresults") # False
rp.crawl_delay("BVT-course") # 5
```

Rules match path *prefixes*: `/search` also blocks `/searchresults`. No `robots.txt` (404) means nothing is disallowed.

Parsing HTML with BeautifulSoup

```
<div class="quote">
<span class="text">"Some quote."</span>
<span>by <small class="author">A. Person</small>
<a href="/author/A-Person">(about)</a></span>
<div class="tags">
<a class="tag" href="/tag/life/">life</a>
</div>
</div>
```

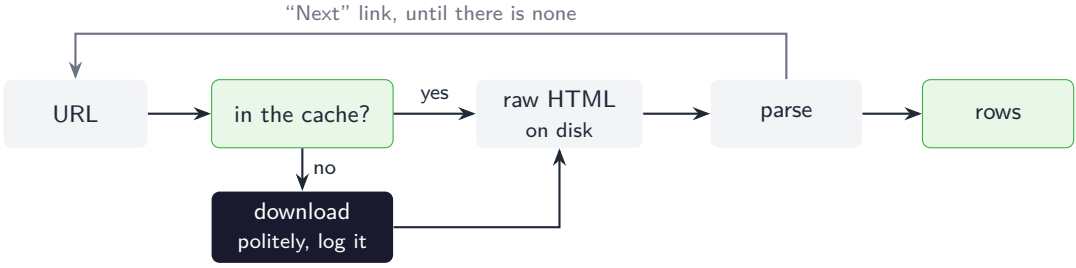
Find the structure with your browser: right-click → **Inspect**.

CSS selectors find tags by name and class:

`div.quote`	each quote
`small.author`	the author
`div.tags a.tag`	tags inside the quote
`li.next a`	the "Next" link
`a[href^='/author/']`	links starting with `/author/`

Practice site: quotes.toscrape.com, made for learning to scrape.

Many Pages: Follow, Save, Then Parse



Separate downloading from parsing. Your parser will have bugs; fix and re-parse without downloading again. The log of URLs and times documents the collection.

Scrapers break silently when a site changes its HTML. Count rows per page, compare two routes to the same data, look at samples.

APIs: OpenAlex and Wikipedia

OpenAlex: a free, open index of scholarly works

- titles, abstracts, venues, citations; a ready-made corpus of abstracts
- `select=`: only the fields you need
- `group_by=publication_year`: counts in one request
- **cursor paging** for more than 100 results
- abstracts arrive as an *inverted index*: word → positions

Since February 2026: a free API key with a daily budget. Keep keys in Colab *Secrets* or an environment variable, never in a shared notebook.

Wikipedia: plain text of any article

```
params = {"action": "query",
"prop": "extracts", "explaintext": 1,
"titles": "Natura 2000",
"format": "json", "formatversion": 2}
polite_get("https://en.wikipedia.org/w/api.php",
params=params)
```

Any language edition: change `en` to `nl`. Wikimedia requires a descriptive `User-Agent`.

Also worth knowing: Tweede Kamer Open Data, CBS Open Data, Guardian and NYT APIs, Common Crawl.

Harder Cases

JavaScript pages

`requests` sees an empty page.

1. Find the JSON behind it: **Inspect → Network → Fetch/XHR**
2. Or drive a real browser: **Playwright**, Selenium

News

Library databases (Nexis Uni) have full texts of most newspapers; Delpher has Dutch papers back to the 1600s.

For web articles: **trafilatura** extracts the main text (Bonus 1).

Social media

Scraping usually breaks the terms; APIs became restricted and expensive (the "post-API age").

Routes now: platform research programs, and the EU **Digital Services Act**, Art. 40: data access for vetted researchers.

In the Notebook

- A checklist before collecting anything
- HTTP basics and a polite request function with retries and backoff
- Reading `robots.txt` with Python (and one of its quirks)
- Scraping a practice site with BeautifulSoup: selectors, pagination, a cache
- OpenAlex: searching, counting per year, cursor paging, rebuilding abstracts
- Wikipedia articles as plain text, combined with TF-IDF from Day 1

`bonus/notebooks/3_web_scraping_and_apis_exercises.ipynb` · 5 exercises · no GPU needed